

	Network Administration <i>Installing and Configuring an FTP Server</i>	2025/2026
---	--	--

Objectives:

- Deploy and configure a secure SFTP server.
- Access and test SFTP services using DNS.

Background / Scenario

FTP (File Transfer Protocol) is a standard protocol for transferring files over a network. It works similarly to HTTP in that it defines a communication language between a client and a server. However, FTP is unencrypted by default, meaning that usernames, passwords, and data are transmitted in plain text. This makes FTP unsuitable for secure environments. FTPS (FTP over SSL/TLS) extends FTP by encrypting the communication channel, similar to how HTTPS secures HTTP traffic. Using FTPS, sensitive data can be securely transferred between hosts while maintaining authentication and confidentiality.

This lab builds on the network namespaces and DNS setup you configured previously, allowing the deployment of a secure FTP server in a Linux namespace while testing remote access from client PCs (PC1 and PC2). It will be great to clean up HTTP and HTTPS before starting the FTPS lab, while keeping DNS and routing working. The main objectives are:

- Deploy and configure an FTPS server in a dedicated namespace
- Configure the server to listen on standard FTP ports with SSL/TLS enabled.
- Integrate FTPS with DNS and allow access from client namespaces using DNS hostnames rather than IP addresses.

In summary, similar to Lab 5, this lab assumes that the DNS server is already configured and reachable at 192.168.30.2, and that PC1 and PC2 can successfully resolve DNS names. For this FTPS lab, mainly we are assuming the following:

- Network topology is already in place (configured in Lab 0).
- Static routing and SNAT are configured and functional (from Lab 3).
- DNS server is operational (Lab 4), currently using the domain lab.local.
- HTTPS web server is running (Lab 5), integrated with the existing network and reachable via its DNS hostname.

This setup ensures that the FTPS server can be securely deployed in its own namespace, while PC1 and PC2 will be able to reach it using a DNS hostname, relying on the existing DNS infrastructure and routing.

Part 1: Verify network configuration

Step 1: Execute the Lab0's script

PCs and Gateway should already have static IPs:

- PC1 → 192.168.10.10/24
- PC2 → 192.168.20.10/24
- Gateway LAN → 192.168.10.1 / 192.168.20.1
- ISP → 209.165.200.225/27

Step 2: Test basic connectivity

#All LAN ↔ Gateway and Gateway ↔ ISP links should work.

```
sudo ip netns exec PC1 ping -c 3 192.168.10.1
```

```
sudo ip netns exec PC2 ping -c 3 192.168.20.1
```

```
sudo ip netns exec GATEWAY ping -c 3 209.165.200.225
```

Step 3: Execute Lab 3's script (SNAT+ Internet access)

#All LAN ↔ Gateway and Gateway ↔ ISP links should work.

```
sudo ip netns exec PC1 ping -c 3 192.168.10.1
```

```
sudo ip netns exec PC2 ping -c 3 192.168.20.1
```

```
sudo ip netns exec GATEWAY ping -c 3 209.165.200.225
```

Step 4: Verify the topology

Network	Purpose
192.168.10.0/24	LAN 1 (PC1)
192.168.20.0/24	LAN 2 (PC2)
192.168.30.0/24	DNS Network

Key devices, DNS is not in the same LAN, so traffic must be routed.

PC1 → 192.168.10.10

PC2 → 192.168.20.10

Gateway → Routes between all networks

DNS Server → 192.168.30.2

Step 5: Execute Lab 4's script (DNS)

Execute the DNS script allowing the creation of DNS server, connecting it to the Gateway, assigning IPs, creating zone files, starting BIND, and configuring PCs to use DNS. Running this script allows deploying the DNS lab quickly and reliably, ensuring that the network, DNS, and routing configurations are applied correctly every time.

Step 6: Test and verification

```
sudo ip netns exec PC1 ping -c 1 192.168.30.2
sudo ip netns exec PC2 ping -c 1 192.168.30.2
sudo ip netns exec PC1 dig PC2.lab.local +short
sudo ip netns exec PC1 dig -x 192.168.30.2 +short
sudo ip netns exec PC1 dig google.com +short
# You can eventually debug as follows. Silent success/OK means configuration is valid
sudo ip netns exec DNS named-checkconf
sudo ip netns exec DNS named-checkzone lab.local /etc/bind/db.lab.local
```

Step 7: Execute Lab 5's script (Webserver with HTTP)

Execute the Lab 5's script to set up the web server within an isolated network namespace, assigns a static IP address, connects it to the Gateway, and installs and starts the server. It ensures that the web server is fully integrated with the existing static networking, routing, and NAT configuration and that it can be accessed by client namespaces using its DNS hostname.

Step 8: Test and verification

```
# Verify DNS is listening
sudo ip netns exec DNS ss -ulnp | grep :53
# Final test
sudo ip netns exec PC1 dig web.lab.test @192.168.30.2 +short
sudo ip netns exec PC1 curl http://web.lab.test1
```

Step 9: Execute Lab 5's script (Webserver with HTTPS)

Execute the script that enables HTTPS and verifies that the HTTPS page is accessible from the PCs.

Step 10: Test and verification

```
# Simple HTML page to confirm everything works.
sudo ip netns exec WEB bash -c 'echo "<h1>Web Server is running (HTTPS)</h1>" >
/var/www/html/index.html'
# Confirms Apache is listening on 443 and SSL is functional
sudo ip netns exec WEB curl -k https://127.0.0.1
# Confirms routing is correct, firewall allows HTTPS, and end-to-end encryption works
sudo ip netns exec PC1 curl -k https://192.168.40.2
sudo ip netns exec PC1 curl -k https://web.lab.test
```

Until this point, the lab environment was progressively built to emulate a realistic network infrastructure with routing, NAT, DNS, and web service.

The next part of the lab will be dedicated in deploying a secure file transfer service using SFTP on the existing web server namespace. This lab extends the previous HTTPS exposure lab by adding secure file exchange capabilities while preserving network isolation and routing through the Gateway.

¹ Killing the old named process and starting a fresh one inside the DNS namespace sometime fixed any problem. Always stop/kill named before restarting, especially in namespaces.

```
sudo ip netns exec DNS pkill named || true
sudo ip netns exec DNS named -g -u root -c /etc/bind/named.conf &
```

Part 2: Install and configure SFTP

Step 1: Variables

```
SFTP_USER="sftpuser"
SFTP_PASS="Lab1234"
WEB_NS="WEB"
SFTP_HOME="/home/$SFTP_USER"
SFTP_DIR="$SFTP_HOME/files"
```

- SFTP_USER: Dedicated user for file transfers
- SFTP_PASS: Password used by students
- WEB_NS: The namespace where the web/SFTP server runs
- SFTP_HOME: User's home directory
- SFTP_DIR: Upload/download directory

Step 2: Install the SSH Server (OpenSSH)

```
# SFTP works on top of SSH
# Install the SSH server inside the WEB namespace
# The web server now supports secure remote connections
sudo ip netns exec $WEB_NS apt update -qq
sudo ip netns exec $WEB_NS apt install -y openssh-server
```

Step 3: Create an SFTP User

```
# Checks if sftpuser already exists, If not, creates a new user without shell access.
# This user will be used only for file transfers
sudo ip netns exec $WEB_NS id $SFTP_USER || \
sudo ip netns exec $WEB_NS adduser --gecos "" --disabled-password $SFTP_USER
# Set the Password
echo "$SFTP_USER:$SFTP_PASS" | sudo ip netns exec $WEB_NS chpasswd
```

Step 4: Configure Secure Chroot Directory

```
# SFTP users are chrooted for security. The home directory must be owned by root.
# Prevents users from accessing the rest of the system.
sudo ip netns exec $WEB_NS mkdir -p $SFTP_DIR
sudo ip netns exec $WEB_NS chown root:root $SFTP_HOME
sudo ip netns exec $WEB_NS chmod 755 $SFTP_HOME
# Upload Directory Permissions. Only sftpuser can read/write files.
# Other users cannot access uploaded data.
sudo ip netns exec $WEB_NS chown $SFTP_USER:$SFTP_USER $SFTP_DIR
sudo ip netns exec $WEB_NS chmod 700 $SFTP_DIR
```

Step 5: Configure SSH for SFTP-Only Access

```
# Applies only to sftpuser. Forces SFTP mode (no SSH shell).
# Disables port forwarding and GUI access
sudo ip netns exec $WEB_NS bash -c 'cat >> /etc/ssh/sshd_config <<EOF

Match User sftpuser
    ChrootDirectory /home/sftpuser
    ForceCommand internal-sftp
    AllowTcpForwarding no
    X11Forwarding no
EOF'
```

Step 6: Start SSH Inside the Namespace

```
# systemctl does not work reliably inside namespaces. SSH is started manually.
# /run/sshd is required for SSH to start correctly.
sudo ip netns exec $WEB_NS mkdir -p /run/sshd
sudo ip netns exec $WEB_NS pkill sshd || true
sudo ip netns exec $WEB_NS /usr/sbin/sshd
```

Step 7: Verify That SFTP is Running

```
# Confirms that SSH/SFTP is listening on port 22.
# If you see output, the service is running correctly.
sudo ip netns exec $WEB_NS ss -tulnp | grep :22
```

Step 8: Connection Instructions

```
# Connect from PC1 or PC2 using, Password: Lab1234
sftp sftpuser@web.lab.test
# Inside SFTP, users connect securely from PC1 or PC2.
# Files are transferred with encryption. Users are restricted to /home/sftpuser/files
cd files
put myfile.txt
ls
get myfile.txt
exit
```

Step 9: Testing and traffic analysis

```
# Connect from PC1 or PC2 using, Password: Lab1234
sudo ip netns exec GATEWAY tcpdump -i veth-gw-web port 22 -w sftp_capture.pcap
# Perform SFTP operations from a client
sudo ip netns exec PC1 bash -c "echo 'Hello from PC1 via SFTP' > myfile.txt"
sudo ip netns exec PC1 sftp sftpuser@192.168.40.2
sftp> put myfile.txt
sftp> get myfile.txt
sftp> exit
# Press Ctrl+C in the tcpdump terminal.
# Open the capture with Wireshark or tcpdump itself.
tcpdump -r sftp_capture.pcap -nn
# You will see SSH packets, but the content of files is encrypted, confirming secure transfer.
```

Step 10: Create a Bash script to automate the SFTP deployment

Create a Bash script (enable_sftp.sh) to automate the deployment of a secure file transfer service (SFTP) in the lab environment. This script assumes that the base network configuration is already in place, including Lab 0 (static IP addressing), Lab 3 (routing and NAT), Lab 4 (DNS configuration), and Lab 5 (HTTPS). Running this script enables a consistent, repeatable, and secure deployment of SFTP, making it easy to verify secure file transfer functionality and to demonstrate the use of SSH-based services in a controlled, namespace-based lab environment.

Step 11: Bash script to clean and reset the SFTP

Create a Bash script (cleanup_sftp.sh) to safely remove the secure file transfer service (SFTP) from the lab environment. This script stops the SSH/SFTP service running in the WEB network namespace, removes the dedicated SFTP user and its associated directories, and cleans the SSH server configuration related to SFTP access. The cleanup process is carefully scoped to ensure that it does not affect the existing network topology, DNS configuration (Lab 4), routing and NAT rules (Lab 3), or other previously deployed services such as the web server (Lab 5). Only components explicitly created for the SFTP service are removed.

Deliverables (To be uploaded on the Platform)

In this lab, you designed and configured an SFTP service inside the isolated WEB network namespace, installed and configured the OpenSSH server, and created a dedicated SFTP user with restricted access. A secure chroot environment was deployed and a dedicated upload directory to ensure that file transfers are performed safely and without access to the rest of the system. The Lab does not modify existing DNS, routing, or NAT configurations. Instead, it integrates the SFTP service seamlessly with the existing lab topology, allowing client namespaces (PC1 and PC2) to securely upload and download files using the server's IP address or DNS hostname.

The main report document (Word/PDF, 10 pages max) must include the following:

- SFTP Server Configuration
 - Namespace & network: SFTP server connected to Gateway with static IP; routing and NAT allow PC1/PC2 to reach it.
 - Explanation: Linux network namespaces emulate isolated servers and clients for realistic, controlled lab testing.
- Verification and Testing
 - Functionality: Upload and download files from PC1/PC2 using the server's IP or DNS hostname.
 - DNS & routing: Confirm hostname resolution and check ip route to ensure correct traffic paths.
- Traffic and security
 - Use tcpdump to verify encrypted SFTP traffic; all transfers are secured with SSH.
- Conclusion
 - The SFTP server is accessible from LAN clients via IP or DNS.
 - Demonstrates DNS usage, proper routing, and secure file transfer using SSH in a controlled network namespace lab environment.

A single compressed file named in the format “lastname.firstname-Lab6.zip”, must be uploaded on the platform (see deadlines) and must include:

- The main report document (Word or PDF)
- All developed Bash scripts